Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai) Bhavan's Campus, Munshi Nagar,
Andheri (West), Mumbai-400058-India

Quick Commerce Inventory Database Design

https://github.com/gitdorah/quick-commerce-database

Requirement Analysis and Database Design Assignment

This assignment designs a relational database for a quick-commerce platform that fulfills orders from nearby dark stores. It identifies requirements, models entities and relationships, maps the ER design to relations, normalizes the schema to 3NF, and supplies MySQL table definitions and sample data. The key design objective is to maintain accurate stock states and prevent two concurrent orders from selling the same units.

Student	Batch	UID	Submission Date
Aryan Kadu	B2	2025300103	30/9/26
Trisha Khandelwal	B2	2025300112	30/9/26
Siddhesh Khanolkar	B2	2025300114	30/9/26

1. Case Study and Scope

A quick-commerce service promises delivery in roughly 10–15 minutes by fulfilling orders from a network of small dark stores located close to customers. Customers browse products, submit orders, and receive fulfillment from one selected store. Each store holds multiple products. A product's stock is tracked by state so the system can distinguish stock that is sellable from stock already reserved, damaged, or moving between locations.

The scope covers customer, order, order item, product, dark store, inventory state, and inventory balance records. Payment processing, delivery partners, address book management, promotions, suppliers, and stock movement history are outside the core schema. Inventory quantities represent current balances; an audit-grade production system would also retain an append-only inventory movement ledger.

2. Requirement Analysis

2.1 Functional requirements

- A customer can place many orders; every order belongs to exactly one customer.
- Each order is fulfilled by exactly one dark store and contains one or more order items.

- Each order item identifies one product and a positive quantity. A product may occur in many order items.
- A dark store can stock many products, and a product can be stocked at many dark stores.
- For every store–product pair, the system records quantities by inventory state: Available, Reserved, Damaged, or In-Transit.
- The database must preserve referential integrity, reject negative inventory and nonpositive order quantities, and prevent duplicate balance rows for the same store, product, and state.

2.2 Non-functional requirements and assumptions

Inventory accuracy should be at least 99%, and the system must prevent overselling during peak concurrency. Availability checks and stock changes must be atomic. The design assumes one currency and a single price per product in this teaching example; production systems should record the unit price actually charged on each OrderItem. A reservation is represented by moving quantity from Available to Reserved within one transaction. Expired or failed reservations must be released by a compensating transaction.

3. Entities and Attributes

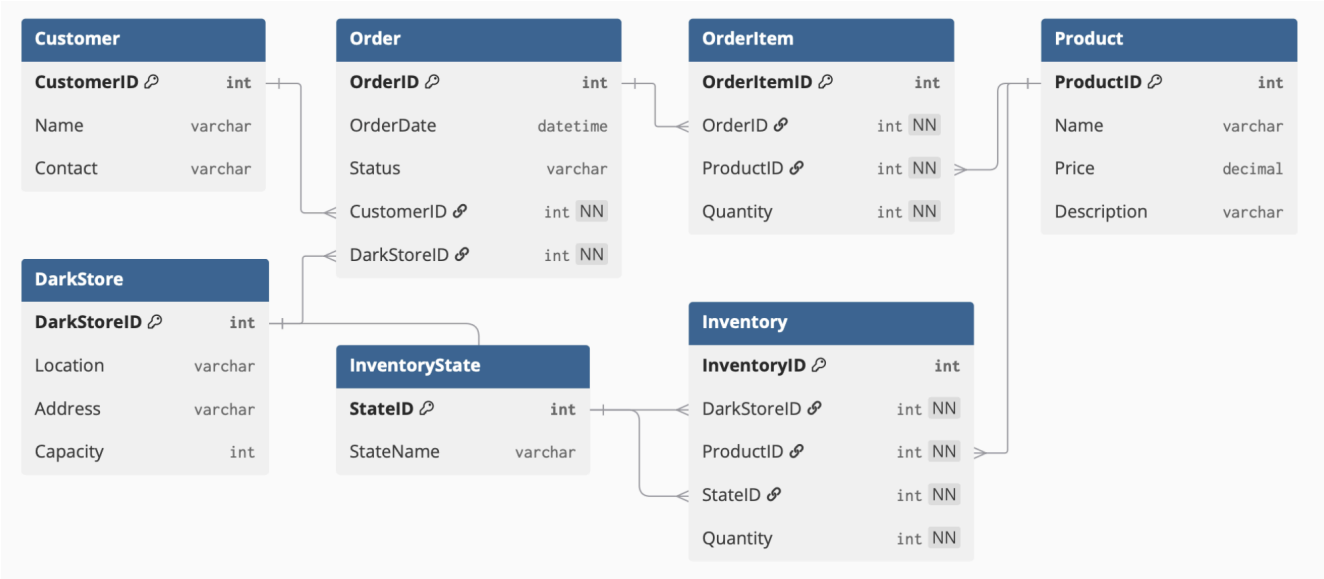
Entity	Key attributes	Purpose
Customer	CustomerID (PK), Name, Contact	Stores customer identity and contact details.
DarkStore	DarkStoreID (PK), Location, Address, Capacity	Represents a fulfillment location.
Product	ProductID (PK), Name, Price, Description	Stores sellable catalog items.
Order	OrderID (PK), OrderDate, Status, CustomerID (FK), DarkStoreID (FK)	Captures a customer purchase and selected store.
OrderItem	OrderItemID (PK), OrderID (FK), ProductID (FK), Quantity	Associative/detail entity listing products in an order.
InventoryState	StateID (PK), StateName	Controlled vocabulary for stock state.
Inventory	InventoryID (PK), DarkStoreID (FK), ProductID (FK), StateID (FK), Quantity	Stores the balance for one store, product, and state.

4. ER Model and Relationships

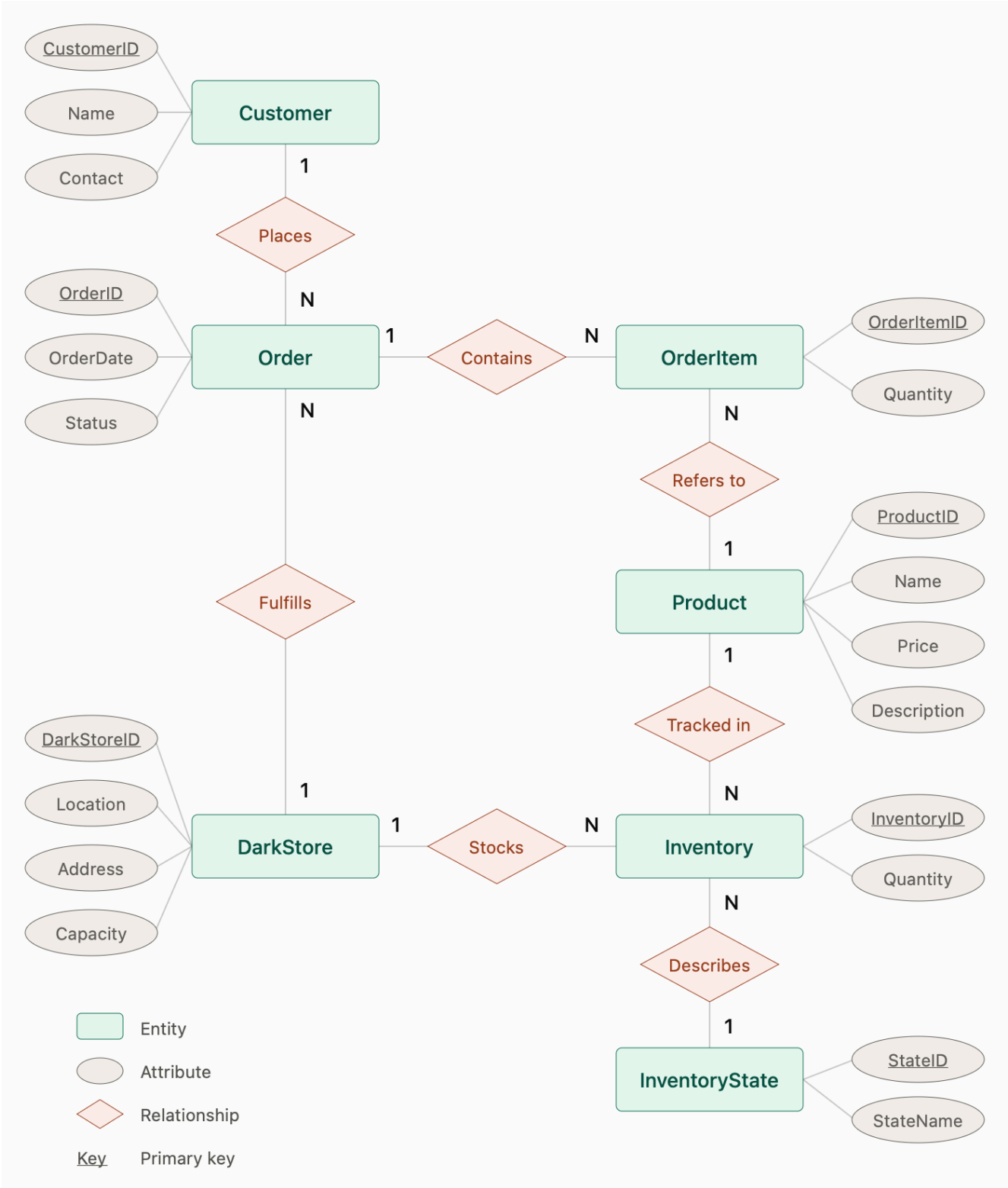
The conceptual ER model is summarized below. Crow’s-foot notation is expressed in words: 1:M means one-to-many. An order item resolves the many-to-many relationship between orders and

products; Inventory resolves the many-to-many relationship between dark stores and products and adds inventory state and quantity.

Relationship	Cardinality	Participation / meaning
Customer — Order	1:M	A customer may place zero or many orders; each order has one customer.
DarkStore — Order	1:M	A store may fulfill many orders; each order is assigned to one store.
Order — OrderItem	1:M	An order has one or more items; each item belongs to one order.
Product — OrderItem	1:M	A product can appear on many order items; each item references one product.
DarkStore — Inventory	1:M	A store has multiple inventory balance rows.
Product — Inventory	1:M	A product has balance rows at different stores and states.
InventoryState — Inventory	1:M	A state labels many balance rows; each row has one state.



Peter Chen Notation ER Diagram:



The Inventory relation is unique on (DarkStoreID, ProductID, StateID). This composite uniqueness rule ensures that a given product has at most one balance row per state at each store. The InventoryID surrogate key is retained for convenient row identification.

5. Relational Schema

CUSTOMER(CustomerID PK, Name, Contact)
DARK_STORE(DarkStoreID PK, Location, Address, Capacity)
PRODUCT(ProductID PK, Name, Price, Description)
INVENTORY_STATE(StateID PK, StateName UNIQUE)
ORDER(OrderID PK, OrderDate, Status, CustomerID FK → CUSTOMER, DarkStoreID FK → DARK_STORE)
ORDER_ITEM(OrderItemID PK, OrderID FK → ORDER, ProductID FK → PRODUCT, Quantity)
INVENTORY(InventoryID PK, DarkStoreID FK → DARK_STORE, ProductID FK → PRODUCT, StateID FK → INVENTORY_STATE, Quantity, UNIQUE(DarkStoreID, ProductID, StateID))

ORDER is a reserved word in MySQL, so SQL statements quote it with backticks. Primary keys uniquely identify rows. Foreign keys enforce valid parent references. NOT NULL and CHECK constraints encode required values and basic domain rules.

6. Normalization

6.1 First Normal Form (1NF)

Each field contains one atomic value. Products in an order are not stored as a comma-separated list; each product is represented by a separate OrderItem row. Inventory state and quantity are also represented as one value per row.

6.2 Second Normal Form (2NF)

Every relation is in 1NF. In the conceptual order line, the natural identifier is (OrderID, ProductID). Quantity depends on the full combination because it describes how many units of that product occur in that order. The implementation uses OrderItemID as a surrogate key and should additionally enforce UNIQUE(OrderID, ProductID) if duplicate lines for the same product are not permitted. Inventory quantity depends on the full natural key (DarkStoreID, ProductID, StateID); the matching UNIQUE constraint prevents duplicate balances.

6.3 Third Normal Form (3NF)

Non-key attributes depend on the key, the whole key, and nothing but the key. Customer contact data is stored in Customer rather than repeated in Order. Product descriptions and prices are stored in Product rather than copied into each stock row. StateName is separated into InventoryState rather than repeated as free text in Inventory. Dark-store address and capacity remain in DarkStore. These separations avoid update, insertion, and deletion anomalies.

For a production order history, add UnitPriceAtOrder to OrderItem so historical totals do not change when the current Product.Price changes. This is a deliberate transaction snapshot, not a reason to duplicate unrelated product attributes.

7. MySQL Implementation

The following script creates the schema and inserts a compact sample dataset. Run it in MySQL 8.0 or a compatible client. The sample IDs are explicit to make the relationships easy to inspect.

```
CREATE DATABASE IF NOT EXISTS QuickCommerceDB;
USE QuickCommerceDB;

CREATE TABLE Customer (
  CustomerID INT PRIMARY KEY AUTO_INCREMENT,
  Name VARCHAR(100) NOT NULL,
  Contact VARCHAR(15) NOT NULL UNIQUE
);
CREATE TABLE DarkStore (
  DarkStoreID INT PRIMARY KEY AUTO_INCREMENT,
  Location VARCHAR(100) NOT NULL,
  Address VARCHAR(255) NOT NULL,
  Capacity INT NOT NULL CHECK (Capacity > 0)
);
CREATE TABLE Product (
  ProductID INT PRIMARY KEY AUTO_INCREMENT,
  Name VARCHAR(100) NOT NULL,
  Price DECIMAL(10,2) NOT NULL CHECK (Price >= 0),
  Description VARCHAR(255)
);
CREATE TABLE InventoryState (
  StateID INT PRIMARY KEY AUTO_INCREMENT,
  StateName VARCHAR(50) NOT NULL UNIQUE
);
CREATE TABLE `Order` (
  OrderID INT PRIMARY KEY AUTO_INCREMENT,
  OrderDate DATETIME NOT NULL,
  Status VARCHAR(30) NOT NULL,
  CustomerID INT NOT NULL,
  DarkStoreID INT NOT NULL,
  FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
  FOREIGN KEY (DarkStoreID) REFERENCES DarkStore(DarkStoreID)
);
CREATE TABLE OrderItem (
  OrderItemID INT PRIMARY KEY AUTO_INCREMENT,
  OrderID INT NOT NULL,
  ProductID INT NOT NULL,
  Quantity INT NOT NULL CHECK (Quantity > 0),
  UNIQUE (OrderID, ProductID),
  FOREIGN KEY (OrderID) REFERENCES `Order`(OrderID),
  FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
CREATE TABLE Inventory (
  InventoryID INT PRIMARY KEY AUTO_INCREMENT,
  DarkStoreID INT NOT NULL,
  ProductID INT NOT NULL,
  StateID INT NOT NULL,
```

```
  Quantity INT NOT NULL CHECK (Quantity >= 0),
  UNIQUE (DarkStoreID, ProductID, StateID),
  FOREIGN KEY (DarkStoreID) REFERENCES DarkStore(DarkStoreID),
  FOREIGN KEY (ProductID) REFERENCES Product(ProductID),
  FOREIGN KEY (StateID) REFERENCES InventoryState(StateID)
);

INSERT INTO Customer (CustomerID,Name,Contact) VALUES
(1,'Aarav Mehta','9876543210'),(2,'Riya Sharma','9823456712'),
(3,'Kabir Shah','9765432189');
INSERT INTO DarkStore (DarkStoreID,Location,Address,Capacity) VALUES
(1,'Andheri','Andheri West, Mumbai',5000),
(2,'Bandra','Bandra East, Mumbai',4500);
INSERT INTO Product (ProductID,Name,Price,Description) VALUES
(1,'Amul Taaza Milk 1L',62.00,'Fresh toned milk'),
(2,'Britannia Bread 400g',45.00,'White sandwich bread'),
(3,'Lay''s Classic Salted 50g',20.00,'Classic salted potato chips');
INSERT INTO InventoryState (StateID,StateName) VALUES
(1,'Available'),(2,'Reserved'),(3,'Damaged'),(4,'In-Transit');
INSERT INTO `Order` (OrderID,OrderDate,Status,CustomerID,DarkStoreID) VALUES
(1,'2026-09-30 09:15:00','Delivered',1,1),
(2,'2026-09-30 09:22:00','Preparing',2,2),
(3,'2026-09-30 09:31:00','Confirmed',3,1);
INSERT INTO OrderItem (OrderItemID,OrderID,ProductID,Quantity) VALUES
(1,1,1,2),(2,1,2,1),(3,2,3,3),(4,3,1,1);
INSERT INTO Inventory (InventoryID,DarkStoreID,ProductID,StateID,Quantity) VALUES
(1,1,1,1,25),(2,1,1,2,3),(3,1,2,1,40),
(4,1,3,1,60),(5,1,3,3,2),(6,2,1,1,20),
(7,2,2,1,30),(8,2,3,1,45),(9,2,3,2,5);
```

8. Preventing Overselling with an Atomic Transaction

A plain read followed later by an update is unsafe: concurrent checkouts may both observe the same available quantity. A conditional update makes the stock check and decrement one database operation. If affected_rows is zero, available stock was insufficient. Reservation is then increased in the same transaction. The application should roll back if either required update fails.

```
START TRANSACTION;
UPDATE Inventory
SET Quantity = Quantity - 2
WHERE DarkStoreID = 1 AND ProductID = 1 AND StateID = 1
  AND Quantity >= 2;
-- Application checks ROW_COUNT() = 1; otherwise ROLLBACK.
UPDATE Inventory
SET Quantity = Quantity + 2
WHERE DarkStoreID = 1 AND ProductID = 1 AND StateID = 2;
-- If the Reserved row may not exist yet, create it safely or seed it at zero.
```

```
COMMIT;
```

The two balance changes must commit or roll back together. In a production implementation, also create/update the order and its items in that transaction, use a consistent lock/update order to reduce deadlocks, and retry deadlock victims safely. The conditional decrement prevents a negative Available balance; transactional atomicity prevents a partial move between Available and Reserved.

9. Sample Queries

```
-- Available quantity of each product at each store
SELECT ds.Location, p.Name, i.Quantity AS AvailableQuantity
FROM Inventory i
JOIN DarkStore ds ON ds.DarkStoreID = i.DarkStoreID
JOIN Product p ON p.ProductID = i.ProductID
JOIN InventoryState s ON s.StateID = i.StateID
WHERE s.StateName = 'Available';

-- Customer orders with line items
SELECT o.OrderID, c.Name, o.OrderDate, p.Name AS Product, oi.Quantity
FROM `Order` o
JOIN Customer c ON c.CustomerID = o.CustomerID
JOIN OrderItem oi ON oi.OrderID = o.OrderID
JOIN Product p ON p.ProductID = oi.ProductID
ORDER BY o.OrderDate, o.OrderID;
```

10. Conclusion

The design models the central quick-commerce entities and separates order details, product catalog data, store information, and inventory states into normalized relations. Primary keys, foreign keys, domain checks, and composite uniqueness constraints protect data integrity. A conditional, transactional stock reservation ensures competing checkouts cannot reserve more units than are available, supporting the business goal of reliable real-time inventory and zero overselling.